



Instituto Tecnológico
de Buenos Aires

HCI - HTML to Composable Instructions TP TLA

Segundo cuatrimestre de 2025

Comisión S - Grupo 105

Integrantes:

Cuneo Gima, Matias

64492

Tabla de contenidos

Tabla de contenidos.....	1
Introducción.....	2
Modelo Computacional.....	2
1. Dominio.....	2
2. Lenguaje.....	3
2.1 Lenguaje de entrada: HTML.....	3
2.2 Lenguaje de salida: Composables de Jetpack Compose.....	3
Implementación.....	4
1. Frontend.....	4
1.1 Análisis léxico.....	4
1.2 Análisis sintáctico.....	4
2. Backend.....	5
2.1 Primera pasada: Reconocimiento de Headers y Footers.....	5
2.2 Segunda pasada: Generación del código Kotlin.....	5
3. Dificultades encontradas.....	6
Futuras Extensiones.....	6
Conclusiones.....	7
Bibliografía.....	7

Introducción

En el desarrollo de aplicaciones modernas, la convergencia entre tecnologías web y móviles se ha vuelto un factor clave para reducir tiempos de implementación y facilitar la reutilización de conocimientos y arquitecturas. Sin embargo, la transición desde interfaces basadas en HTML hacia ecosistemas móviles nativos, como Android, suele implicar un proceso manual y propenso a inconsistencias, especialmente cuando se busca mantener la estructura semántica y la lógica de presentación originales.

Con el objetivo de optimizar este proceso, se presenta un compilador/traspilador que convierte código HTML en funciones Composable de Jetpack Compose, el moderno framework declarativo para interfaces en Android. Esta herramienta permite que un equipo con experiencia en desarrollo web pueda migrar rápidamente sus diseños y componentes hacia una aplicación móvil nativa, manteniendo la coherencia visual y reduciendo significativamente el esfuerzo de reimplementación.

El sistema desarrollado actúa como una capa de transformación que interpreta la estructura del documento HTML (etiquetas, atributos y jerarquía) y la traduce a composables equivalentes en Compose, respetando patrones declarativos y buenas prácticas del entorno Android. De esta manera, se propone una solución eficiente, extensible y orientada a mejorar la productividad en contextos donde se necesiten ambas plataformas web como mobile.

Modelo Computacional

1. Dominio

El dominio del proyecto está en la intersección entre el desarrollo web y el desarrollo móvil nativo, específicamente en el contexto de la migración de interfaces HTML hacia la plataforma Android utilizando Jetpack Compose. El sistema opera sobre documentos HTML, cuyo objetivo principal es describir una estructura de interfaz mediante etiquetas, atributos y un modelo jerárquico basado en nodos.

Dentro de este dominio, el compilador se encarga de procesar:

- Estructuras jerárquicas (árbol DOM) compuestas por elementos anidados.
- Etiquetas HTML estándar que representan contenedores, bloques, elementos inline y componentes semánticos.
- Atributos relevantes para convertir a propiedades de Jetpack Compose.
- Contenido textual.

El resultado final del proceso debe situarse dentro del dominio del desarrollo nativo Android, donde la interfaz se representa mediante funciones Composable declarativas que expresan la estructura visual como un árbol de llamadas a funciones, manteniendo correspondencia con la semántica del HTML fuente.

2. Lenguaje

El compilador trabaja con dos lenguajes bien diferenciados:

2.1 Lenguaje de entrada: HTML

El lenguaje de entrada es HTML5, un lenguaje de marcado orientado a describir la estructura lógica y visual de un documento. Para este proyecto, el subconjunto de HTML considerado incluye:

- Etiquetas estructurales (div, header, footer, etc.).
- Etiquetas de texto (p, span, h1-h6).
- Etiquetas de control visual básico (img, button, input con restricciones).
- Atributos estándar (globales y algunos específicos).

La semántica del HTML en este contexto se interpreta únicamente desde su composición visual, sin ejecutar scripts ni comportamientos dinámicos propios del DOM. Se asume un árbol HTML bien formado y se ignoran elementos no mapeables a composables estándar.

2.2 Lenguaje de salida: Composables de Jetpack Compose

El lenguaje de salida está compuesto por funciones Composable escritas en Kotlin y diseñadas bajo el paradigma declarativo de Jetpack Compose. Sus características más relevantes para este modelo son:

- Cada elemento de la UI se representa como una función anotada con `@Composable`, lo que exige mantener un mapeo directo desde las etiquetas HTML hacia composables equivalentes o personalizados.
- La estructura de composición se expresa a través de llamadas anidadas, que reemplazan la jerarquía DOM por un árbol de composables.
- Los atributos del HTML se traducen a parámetros de función, modificadores (Modifier), o estilos declarativos siguiendo las convenciones de Compose.
- La salida debe ser código Kotlin válido, legible y extensible, permitiendo su incorporación inmediata dentro de un proyecto Android.

Este lenguaje destino introduce restricciones semánticas importantes: la UI debe ser declarativa, sin mantener estado interno mutable, y los estilos deben adaptarse al sistema de modificadores propio de Compose. El modelo computacional garantiza que la traducción preserve la estructura visual, pero no necesariamente los estilos exactos o el comportamiento dinámico asociado al HTML original.

Implementación

La implementación del compilador se divide en dos etapas principales: Frontend y Backend. Cada una cumple un rol específico dentro del proceso de traducción, desde la lectura y análisis del código HTML hasta la generación del código Kotlin con funciones Composable.

1. Frontend

El frontend está implementado utilizando las herramientas de: Flex para el análisis léxico y Bison para el análisis sintáctico. Su función principal es recibir el documento HTML de entrada, reconocer su estructura y construir un Árbol de Sintaxis Abstracta (AST) que represente la jerarquía lógica del documento.

1.1 Análisis léxico

Flex se encarga de identificar los tokens relevantes del lenguaje HTML, tales como:

- nombres de etiquetas,
- atributos y valores,
- contenido textual.

El analizador léxico filtra elementos irrelevantes, como espacios no significativos o saltos de línea, y genera un flujo de tokens estructurado para el parser.

1.2 Análisis sintáctico

Bison recibe los tokens producidos por Flex y aplica la gramática definida para HTML (acotada al subconjunto soportado por el compilador). Esta gramática describe cómo se construyen los elementos HTML y cómo se organizan jerárquicamente.

A través de sus reglas de producción, Bison:

- Reconoce elementos simples y compuestos.
- Valida el correcto anidamiento de etiquetas.
- Construye nodos específicos para el AST, asignando atributos, contenido textual y listas de hijos.

El resultado final es un AST limpio, estructurado y simplificado, que representa la semántica esencial del documento HTML sin detalles irrelevantes para la etapa de generación de código.

2. Backend

El backend es responsable de transformar el AST generado por el frontend en el código Kotlin final, utilizando funciones Composable equivalentes a las estructuras HTML originales.

A diferencia del frontend, que es directo y de una pasada, el backend implementa un enfoque de doble pasada sobre el AST. Esta estrategia permite resolver componentes globales y luego generar código final limpio y coherente.

2.1 Primera pasada: Reconocimiento de Headers y Footers

En la primera recorrida, el backend visita todo el árbol en busca de elementos que deben tratarse de manera especial, principalmente:

- encabezados globales (Headers)
- pies de página (Footers)

Estos elementos suelen requerir una traducción distinta, ya que generan composables específicos.

Durante esta pasada, el backend:

- Identifica nodos `<header>` y `<footer>`.
- Registra su posición y contenido.
- Construye estructuras auxiliares que servirán para la generación posterior.

No se produce salida durante esta etapa.

2.2 Segunda pasada: Generación del código Kotlin

La segunda pasada del backend realiza la traducción efectiva de los nodos HTML al lenguaje destino: Kotlin con Jetpack Compose.

Para cada nodo del AST, el backend:

- Determina la correspondencia entre la etiqueta HTML y su composable equivalente (por ejemplo, `div` a `Column`, `span` a `Text`, etc.).
- Traduce los atributos HTML a parámetros, modificadores o estilos de Compose.
- Genera funciones `@Composable` para los elementos principales.

- Respetar la estructura jerárquica del AST, produciendo llamadas anidadas que preservan la composición visual.
- Insertar las referencias a Headers y Footers previamente detectados, siguiendo un orden lógico.

El resultado final es un archivo Kotlin válido, bien formateado y listo para integrarse en un proyecto Android. El backend asegura que la salida mantenga la correspondencia estructural con el HTML original, adaptándola al paradigma declarativo de Jetpack Compose.

3. Dificultades encontradas

Las principales dificultades surgieron durante la implementación del frontend, específicamente en el diseño y la definición de la gramática utilizada por el analizador sintáctico.

En esta etapa se presentaron problemas asociados al tratamiento de etiquetas HTML que pueden incluir simultáneamente contenido interno y atributos opcionales. Un caso representativo es el de elementos como `<div>`, cuya estructura admite la presencia o ausencia de atributos (por ejemplo, `style`) y, a su vez, permite contener otras etiquetas anidadas o texto.

La combinación de estas variantes generó ambigüedades gramaticales que derivaron en que no se permitieran dar estas condiciones en la entrada del compilador, eliminando los atributos de las etiquetas con contenido.

Futuras Extensiones

Como trabajo futuro, se propone ampliar las capacidades del compilador mediante las siguientes líneas de evolución:

- Soporte integral de atributos para todas las etiquetas actualmente reconocidas, garantizando una traducción más precisa y completa hacia parámetros y modificadores de Jetpack Compose.
- Interpretación y mapeo de estilos, permitiendo traducir propiedades visuales del HTML a estructuras equivalentes en Compose.
- Incorporación de un conjunto más amplio de etiquetas HTML, con el objetivo de aumentar la cobertura del lenguaje de entrada y mejorar la flexibilidad del compilador.

Estas extensiones permitirían incrementar la expresividad del sistema y acercar la herramienta a un uso productivo en escenarios reales de migración y prototipado.

Conclusiones

El desarrollo de este compilador de HTML a funciones Composable de Android permitió explorar en profundidad los procesos fundamentales de análisis léxico, sintáctico y generación de código dentro del contexto de un traspilador orientado a la migración de interfaces entre plataformas. A lo largo del proyecto se logró implementar un frontend funcional basado en Flex y Bison, capaz de construir un AST coherente a partir de documentos HTML, así como un backend que, mediante una doble pasada, genera código Kotlin válido y estructuralmente consistente con el documento original.

Los objetivos propuestos fueron alcanzados de manera satisfactoria. El sistema resultante constituye una base sólida para futuras ampliaciones y demuestra la viabilidad de automatizar parcialmente la transición desde arquitecturas web hacia entornos móviles nativos utilizando Jetpack Compose. Asimismo, el proceso permitió afianzar conocimientos de diseño de compiladores, gramáticas y estructuras de traducción declarativas.

Finalmente, expreso mi gratitud por la oportunidad de haber llevado adelante este trabajo y mi conformidad con los resultados obtenidos. La implementación realizada no sólo cumple con los requisitos planteados, sino que también abre el camino a mejoras y extensiones que podrán consolidarse en desarrollos futuros.

Bibliografía

Finalmente, se incluye la documentación relevante al contenido del informe.

Android : <https://developer.android.com/develop?hl=es-419>

Kotlin : <https://kotlinlang.org/docs/home.html>

Jetpack Compose :

<https://developer.android.com/develop/ui/compose/documentation?hl=es-419>